

Technical Guide for Road Extraction ArcGis Integration

2026-06-05

Contents

1	GitHub Repository	3
2	Guide Overview	3
3	General Function	3
3.1	Environment Setup	4
3.2	GUI	6
3.3	Standalone ONNX Inference Script	9
4	Troubleshooting	10

1 GitHub Repository

All required source files, scripts and ArcGIS toolbox files are available through the official GitHub repository:

`https://github.com/majrblais/ArcGUI`

Clone or download the repository before configuring the environment.

The trained ONNX model weights are provided separately through a OneDrive download link due to file size limitations on GitHub. After downloading the ONNX model file, place it inside the repository root directory.

2 Guide Overview

This document provides a practical deployment and integration guide for the proposed road segmentation workflow within ArcGIS Pro. The complete technical report, including methodology, experiments and quantitative evaluation results, is provided separately. This guide focuses primarily on deployment, environment setup, GUI usage, ONNX inference and troubleshooting.

3 General Function

As mentioned in the technical guide, the model is based on a deep learning segmentation algorithm which learns from annotated data. The model has already been trained on similar imagery and can therefore be used directly for inference. However, like most deep learning models, it may still suffer from limitations related to data variety, annotation consistency and image quality. For this reason, the proposed tool should be considered a decision-support tool rather than a complete replacement for expert verification. The model does not require a GPU for inference, but GPU acceleration is strongly recommended to improve speed and efficiency. The first step of the workflow is to define the geographic region or regions that must be processed. The updated implementation supports either a single extent layer or multiple extent layers. Each extent layer represents one area of interest and is processed independently by the tool. If multiple extent layers are selected, the tool processes them sequentially one by one using the same model and parameter configuration. This approach is useful when several cells, management units or regions must be predicted in the same execution. Each extent can be defined from an existing polygon layer, such as a predefined cell or area of interest. If no extent layer is available, the user can manually create one by drawing a polygon or rectangular area in ArcGIS Pro. The geometry of the selected extent is used to define the imagery extraction region and to clip the final prediction back to the original area. This ensures that the final result corresponds only to the intended processing region. Once an extent has been selected, the tool exports imagery directly from the selected region using the ArcGIS *Export Training Data for Deep Learning* tool. The updated workflow no longer requires the

user to manually create a fishnet before prediction. Instead, image tiles are automatically extracted from the selected extent using the tile size, stride and cell size specified in the GUI. For the current model, the necessary tile size is 1024×1024 pixels with a cell size of 2 map units. This configuration matches the training and deployment conditions of the final ONNX model. After tile extraction, each exported tile is preprocessed using the same ImageNet normalization applied during training. The ONNX model then predicts a binary road mask for each tile. The predicted tiles are converted into georeferenced rasters and mosaicked together to reconstruct the complete prediction for the extent. The mosaic is then clipped back to the original extent geometry and cleaned so that background pixels are represented as NoData. For multi-extent processing, each final clean prediction is named using the original extent layer name. For example, if the selected extent layer is named `cellules_469`, the final clean prediction is saved as `cellules_469_roads_prediction_clean.tif`. All final clean predictions are also copied into a shared `final_predictions` folder within the selected output directory. This makes it easier to retrieve the final outputs without searching through the temporary export and prediction folders created for each extent. Only the final clean prediction raster is intended to be used as the final deliverable. The intermediate exported tiles, PNG predictions, temporary raster predictions, raw mosaics and clipped mosaics can be deleted after processing if disk space is limited.

3.1 Environment Setup

As this method is based on Python and requires several external libraries, it is necessary to configure a dedicated Python environment before deployment. Although GPU acceleration is not strictly required, the use of a GPU is highly recommended as it significantly improves inference and processing speed compared to CPU-only systems. NVIDIA GPUs are particularly recommended due to their compatibility with CUDA and ONNX Runtime GPU acceleration. ArcGIS Pro is also required, along with an active subscription, as this workflow depends on tools and imagery services unique to ArcGIS. Once ArcGIS Pro is installed and activated, the Python environment must be configured properly. ArcGIS Pro already provides built-in Python environments, which serve as a strong starting point for deployment. However, installing additional packages directly through ArcGIS can sometimes be difficult due to dependency conflicts and permission restrictions. For this reason, the use of the Anaconda Prompt is recommended for easier environment management and package installation. Several installation guides for Anaconda are available online and can be found at: <https://www.anaconda.com/download> Once installed, Conda can be used to access and manage the Python environments created by ArcGIS Pro, enabling efficient package installation and updates. The first step is to open ArcGIS Pro, navigate to the *Package Manager*, and clone the default environment in order to create a modifiable version. This cloned environment allows additional libraries to be installed without modifying the default ArcGIS configuration. In this guide, the cloned environment is named `torch_roads`, as illustrated in Figure 1.

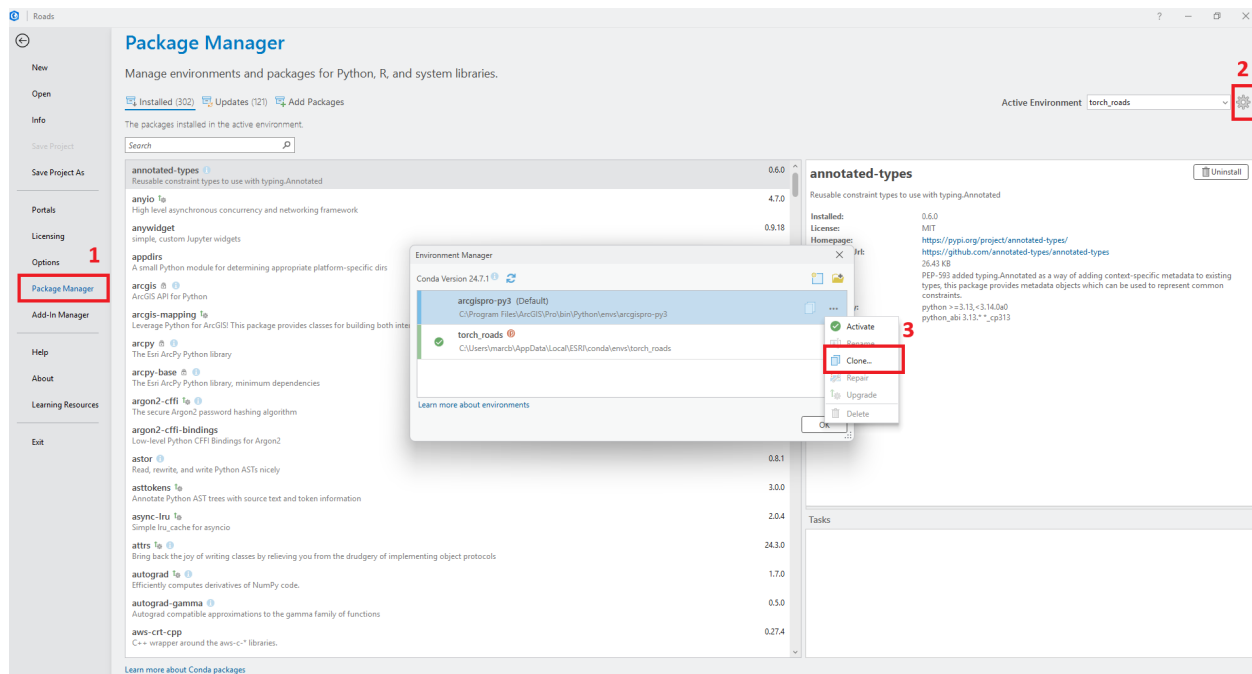


Figure 1: Cloning the ArcGIS Python environment using the Package Manager.

Once the environment has been cloned successfully, open the Anaconda Prompt and activate the environment using the following command:

```
conda activate torch_roads
```

After activation, install the required libraries and dependencies using the following command:

```
git clone https://github.com/majrblais/ArcGUI
cd ArcGUI
pip install -r requirements.txt
```

The exact CUDA and ONNX Runtime versions may vary depending on the installed GPU drivers, CUDA compatibility and ArcGIS configuration. If installation problems occur, users are encouraged to first attempt installation using the most recent package versions. If incompatibilities persist, users may need to install specific CUDA-compatible versions depending on their system configuration and GPU driver version. Once all packages are installed, it is strongly recommended to verify that ONNX Runtime correctly detects and uses the GPU. The check.py verification script checks whether CUDAExecutionProvider is active and performs a small dummy inference using the provided ONNX model.

If the script completes successfully and the CUDAExecutionProvider is listed as active, then ONNX Runtime is correctly configured to use GPU acceleration. If the script fails or ONNX Runtime falls back to CPU execution, the issue is generally related to missing DLL paths, incompatible CUDA versions or incorrect environment configuration. In many cases, these issues can be resolved by ensuring that the correct environment is activated and that all CUDA-related packages are installed properly. If additional issues are encountered

during installation or verification, refer to the troubleshooting section at the end of this guide. Most problems can be resolved by verifying package versions, updating GPU drivers, reinstalling CUDA-related dependencies or correcting missing DLL paths. AI-assisted tools such as ChatGPT may also help diagnose environment configuration issues and dependency conflicts based on the generated error messages.

3.2 GUI

The first implementation consists of a graphical user interface (GUI) integrated directly within ArcGIS Pro. This implementation allows users to perform large-scale road extraction without requiring manual Python execution or command-line interaction. The complete implementation requires only three files: `roadsegmentation_tool.pyt`, `roadsegmentation.python.py` and the provided ONNX model file (`best_unet_resnet101_norm.1024.onnx`). The `.pyt` file corresponds to the ArcGIS Python toolbox interface, while the `.py` file contains the underlying processing and inference pipeline. These two files are found under the same folder and must be left as such when loading the toolbox in ArcGIS. The ONNX file contains the trained deep learning model used for road segmentation. The toolbox implementation automatically exports image tiles from ArcGIS, performs ONNX inference and reconstructs the final prediction raster. The updated toolbox also supports multiple input extent layers, allowing several regions or cells to be processed sequentially during a single execution.

To add the toolbox into ArcGIS Pro, the *Add Toolbox* option must be used. This option can be accessed by right-clicking within the Catalog pane and selecting *Add Toolbox*. The user must then select the provided `pyt` file where it was originally extracted with the `py` file. Figure 2 illustrates the process of importing the Python toolbox into ArcGIS Pro.

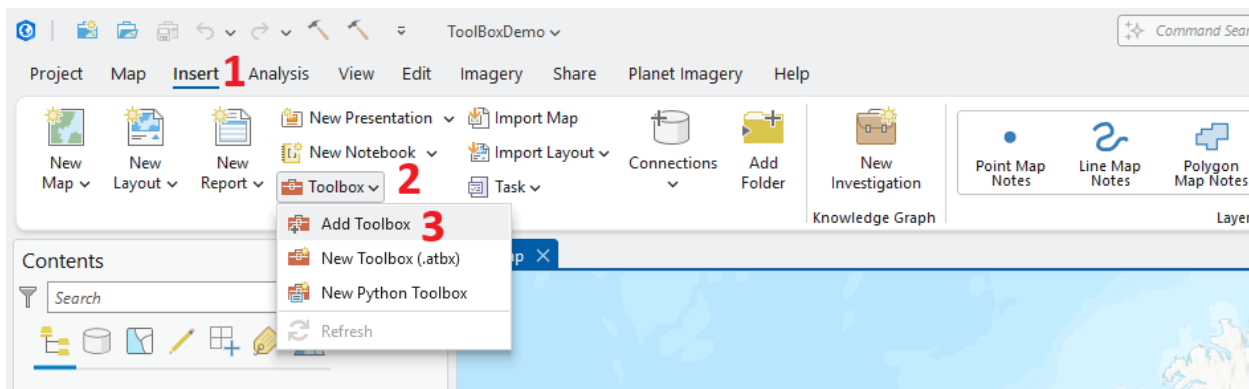


Figure 2: Adding the Python toolbox into ArcGIS Pro using the *Add Python Toolbox* option.

Once imported, the GUI provides multiple configurable parameters for the segmentation pipeline. Figure 3 presents the available GUI options.

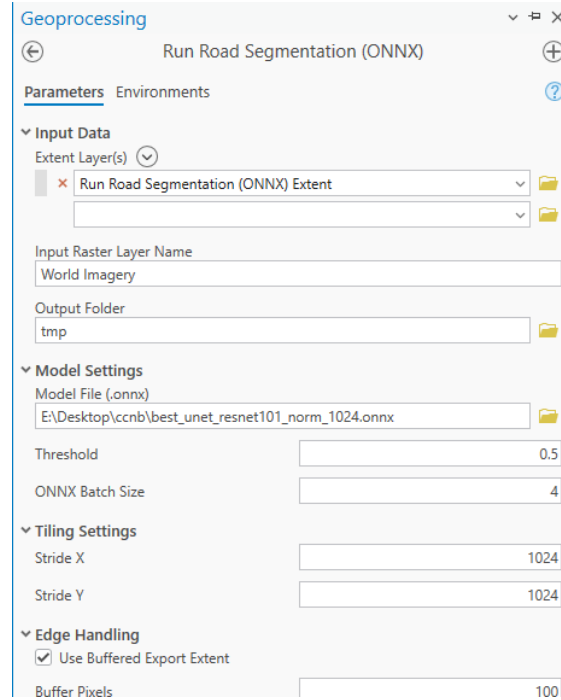


Figure 3: GUI parameters and options for the road segmentation pipeline.

The main GUI parameters are summarized as follows:

- **Extent Layer:** This parameter defines the geographic region or regions processed by the model. The implementation supports one or multiple polygon extent layers, which are processed sequentially using the same configuration parameters. This functionality is useful for predicting multiple cells or disconnected regions without repeatedly launching the tool. Any polygon feature layer can be used as an extent layer. If a custom rectangular region is required, ArcGIS Pro also provides a *Draw Extent* option within the Environment tab. For each processed extent, the final prediction raster is clipped back to the original extent geometry after inference.
- **Input Raster Layer:** This parameter corresponds to the imagery source used for prediction. In most cases, the *World Imagery* layer available within ArcGIS Pro is used. Historical imagery from the Wayback imagery service may also be used for temporal analysis applications. The model expects RGB visible-spectrum imagery as input and was trained using aerial and satellite imagery.
- **Output Folder:** This parameter defines the directory where temporary and final prediction files are stored. For each input extent, the toolbox automatically creates a dedicated subfolder using the original extent layer name. These subfolders contain exported image tiles, intermediate prediction rasters and temporary mosaics generated during processing. A shared `final_predictions` folder is also created inside the selected output directory. Only the final clean raster for each extent is copied into this folder. For example, if the extent layer is named `cellules_469`, the final output

is automatically saved as `cellules_469_roads_prediction_clean.tif`. This naming convention simplifies the association between predictions and their corresponding input regions. Intermediate products such as buffered mosaics and temporary clipped rasters are intended solely for internal processing and can be removed after execution if only the final predictions are required.

- **Model File:** This parameter corresponds to the ONNX segmentation model used during inference. The current implementation supports the provided U-Net model with a ResNet101 encoder trained using ImageNet normalization. The model is loaded automatically using ONNX Runtime with CUDA acceleration when available.
- **Threshold:** This parameter controls the probability threshold used to convert predicted logits into binary road masks. Typical values range between 0.4 and 0.6. Lower threshold values generally increase recall by detecting more potential roads but may also introduce additional false positives. Higher threshold values typically produce cleaner predictions but may remove smaller or uncertain roads. A threshold value of 0.5 was found to provide a balanced trade-off between precision and recall and is therefore used by default.
- **Batch Size:** This parameter controls the number of image tiles processed simultaneously during ONNX inference. The default value is 4. Systems with larger GPU memory capacities may use larger batch sizes to improve inference speed. Conversely, systems with limited GPU memory may require smaller batch sizes to avoid memory allocation issues. The implementation automatically handles incomplete final batches during inference.
- **Stride Parameters:** The stride parameters control the spacing between exported image tiles. Using a stride equal to the tile size produces non-overlapping tiles and maximizes processing speed. Reducing the stride introduces overlap between neighboring tiles. Although overlapping extraction increases computational cost and export time, it may improve prediction consistency near tile boundaries. The implementation fully supports overlapping predictions and automatic mosaicking operations.
- **Edge Handling:** This option addresses artifacts caused by image extraction near processing boundaries. Border tiles may contain incomplete or black regions depending on the extraction geometry. To reduce this issue, the implementation supports buffered export extents. A temporary buffer is added around each original processing extent before tile extraction. After prediction and mosaicking, the final raster is clipped back to the original extent geometry. A buffer size of approximately 100 pixels was found to significantly reduce edge artifacts and improve prediction quality near processing boundaries.
- **Fixed Parameters:** Several parameters remain fixed internally to ensure compatibility with the trained ONNX model. These include:
 - Tile size: 1024×1024

- Cell size: 2
- Model input size: 1024×1024

These values correspond directly to the configuration used during model training and validation. Changing these parameters may produce invalid predictions or inference dimension mismatches. Therefore, they are intentionally fixed within the GUI implementation.

3.3 Standalone ONNX Inference Script

In addition to the ArcGIS GUI implementation, a standalone Python inference script was also developed. This implementation provides a lightweight alternative for users who only require direct prediction on image folders without GIS integration. The script performs ONNX inference on RGB images and automatically saves binary road segmentation masks into an output directory. Unlike the ArcGIS implementation, this standalone version does not require spatial layers, raster mosaicking or georeferencing operations. Instead, it focuses exclusively on image-based inference using the trained ONNX segmentation model. The implementation begins by configuring the CUDA environment required for ONNX Runtime GPU execution. Several NVIDIA CUDA dynamic-link library (DLL) folders are automatically detected and added to the system path before importing ONNX Runtime. This step is particularly important when using ArcGIS Pro Conda environments, as CUDA DLLs installed through Python packages are not always detected automatically. The implementation dynamically scans all NVIDIA package directories and registers their `bin` folders using `os.add_dll_directory`. This process ensures that ONNX Runtime can correctly access CUDA, cuDNN, cuBLAS, cuFFT and related GPU dependencies. Without this step, ONNX Runtime may silently fall back to CPU inference despite CUDA being installed correctly. After configuring the CUDA environment, the implementation loads the ONNX segmentation model using ONNX Runtime. The model is initialized with both the `CUDAExecutionProvider` and `CPUExecutionProvider`. The implementation then verifies that the CUDA provider is active before continuing execution. If CUDA is unavailable, the script raises an error to prevent unintended CPU-only inference. This validation step ensures that GPU acceleration is functioning correctly prior to prediction. The script processes all images located within the specified input directory. Multiple image formats are supported, including PNG, JPEG, TIFF and JPG images. Each image is loaded using the Pillow library and converted into RGB format. The images are then resized to the model input dimensions of 1024×1024 pixels. This resizing operation ensures compatibility with the trained U-Net ResNet101 segmentation model. Before inference, the input images undergo normalization using ImageNet statistics. Pixel intensities are first scaled to the $[0, 1]$ range. The images are then normalized using the ImageNet mean values $[0.485, 0.456, 0.406]$ and standard deviation values $[0.229, 0.224, 0.225]$. These normalization parameters match those used during model training and are therefore required for proper inference performance. The normalized images are subsequently rearranged into channel-first tensor format before being

provided to the ONNX model. The ONNX model produces raw segmentation logits for each image. These logits are transformed into probabilities using the sigmoid activation function. The resulting probability map is then converted into a binary segmentation mask using a configurable threshold parameter. Pixels above the threshold are classified as roads, while pixels below the threshold are classified as background. The default threshold value is set to 0.5, which provides a balanced trade-off between precision and recall. Once prediction is completed, the binary segmentation mask is resized back to the original image dimensions. Nearest-neighbor interpolation is used during resizing to preserve binary mask integrity. The final prediction masks are saved automatically into the output directory using PNG format. Each prediction file uses the original filename with an additional suffix indicating that it corresponds to a predicted segmentation mask. The standalone implementation provides a significantly simpler deployment pipeline compared to the ArcGIS GUI implementation. It is particularly useful for rapid testing, batch inference on image datasets and lightweight deployment scenarios where GIS integration is unnecessary. Because the implementation relies exclusively on ONNX Runtime and standard Python libraries, it can also be integrated easily into external applications or automated processing pipelines.

4 Troubleshooting

Although the proposed implementation was designed to simplify deployment and large-scale road extraction, several issues may still occur depending on the user environment, ArcGIS installation, CUDA configuration or GPU compatibility. Most problems are related to Python environments, CUDA dependencies, ONNX Runtime configuration or ArcGIS environment management. One of the most common issues occurs when cloning the default ArcGIS Python environment. In some cases, ArcGIS Pro may fail to clone the environment through the Package Manager interface. This problem is often caused by insufficient permissions, corrupted Conda metadata or environment conflicts. Restarting ArcGIS Pro with administrator privileges may resolve the issue. If the issue persists, the cloned environment can alternatively be created directly using the Anaconda Prompt. Users should also verify that sufficient disk space is available, as ArcGIS environments can require several gigabytes of storage. Another common issue involves missing Python libraries or failed package installations. ArcGIS environments can occasionally conflict with certain package versions, particularly GPU-related libraries. For this reason, package installation should preferably be performed through the Anaconda Prompt after activating the cloned ArcGIS environment. If installation errors occur, users should first verify that the correct environment is activated using:

```
conda activate <environment_name>
```

Installing packages into the wrong environment is a frequent source of errors. Users should also ensure that both Conda and Pip are updated before installation. In some cases, restarting the environment or reinstalling the packages may resolve dependency conflicts. CUDA

and ONNX Runtime configuration issues are among the most common deployment problems. ONNX Runtime may silently fall back to CPU execution even when a GPU is available. This behaviour generally occurs when CUDA DLLs cannot be located properly by the environment. The provided implementation dynamically registers NVIDIA DLL folders using `os.add_dll_directory` before importing ONNX Runtime. Without this step, ONNX Runtime may fail to load CUDA dependencies correctly. Users should always verify that the `CUDAExecutionProvider` is active before performing inference. Another frequent issue involves version incompatibilities between CUDA, cuDNN and ONNX Runtime. Different ONNX Runtime GPU versions may require specific CUDA and cuDNN versions. If the versions are incompatible, errors such as:

```
LoadLibrary failed with error 126
```

or

```
Failed to create CUDAExecutionProvider
```

may occur. In these situations, reinstalling the NVIDIA runtime libraries using Pip often resolves the issue. Users should ensure that all CUDA-related Python packages are installed within the same environment. It is also recommended to avoid mixing Conda-installed and Pip-installed CUDA libraries whenever possible. GPU driver issues may also prevent CUDA acceleration from functioning correctly. Users should verify that their NVIDIA drivers are updated and compatible with the installed CUDA version. Older GPUs may not support newer CUDA versions required by recent ONNX Runtime releases. In such cases, either the GPU drivers or the ONNX Runtime version may need to be adjusted. If CUDA acceleration cannot be enabled, the implementation can still run using CPU inference, although prediction speed will be substantially slower. Memory-related errors may occur during inference when the selected batch size is too large for the available GPU memory. This typically results in CUDA out-of-memory errors or sudden application crashes. Reducing the ONNX batch size generally resolves the issue. For systems with limited GPU memory, batch sizes of 1 or 2 may be required. Conversely, systems equipped with larger GPUs may benefit from larger batch sizes to improve inference speed. Tile extraction and raster export issues may also occur within ArcGIS Pro. These problems are often caused by invalid extents, unavailable imagery layers or insufficient temporary storage space. When using World Imagery or Wayback imagery, the extraction process depends on remote ArcGIS servers and internet connectivity. Slow internet speeds or unstable connections may therefore affect export performance or cause incomplete tile extraction. Restarting ArcGIS Pro or reducing the processed region size may help resolve these issues. Prediction artifacts near image boundaries can occur when no overlap or buffering is used during tile extraction. These artifacts often appear as black borders or incomplete predictions near tile edges. To reduce these issues, the buffered export extent option should be enabled. A buffer size of approximately 100 pixels is generally recommended for most applications. Users may also encounter slow processing speeds when large geographic regions are processed. Several factors influence deployment speed, including GPU capability, storage speed, internet connectivity and batch size configuration.

The largest bottleneck is often image extraction from World Imagery rather than the ONNX inference itself. Inference using ONNX Runtime with CUDA is generally extremely fast compared to raster export and mosaicking operations. Using faster GPUs, SSD storage and optimized batch sizes can significantly improve overall deployment efficiency. If predictions appear inaccurate or inconsistent, users should first verify that the input imagery matches the type of imagery used during training. The proposed model was trained primarily on visible-spectrum RGB imagery with a spatial resolution of approximately 2 meters per pixel. Significant differences in image resolution, compression quality, seasonal appearance or geographic context may reduce prediction quality. Similarly, annotation inconsistencies may also affect prediction interpretation, particularly for paved roads and highways which were not consistently included as target classes during annotation. When using the multi-extent option, users should ensure that all selected extent layers are valid polygon layers and that their names are unique. The tool uses the original extent layer name to create the final output filename. Duplicate or invalid layer names may therefore lead to confusing outputs or overwritten files if they are not sanitized properly. If the tool fails for one extent, the remaining extents can still be processed, but the failed extent should be checked individually afterward. Common causes of failure include invalid geometry, missing spatial reference information, inaccessible imagery and insufficient disk space. Finally, if persistent issues occur that cannot be resolved through standard troubleshooting procedures, users are encouraged to consult online documentation related to ArcGIS Pro, ONNX Runtime, CUDA or NVIDIA drivers. Modern AI-assisted tools such as ChatGPT may also help identify environment configuration issues, dependency conflicts or CUDA-related errors based on the displayed error messages.